

ITAS170 — Lab 3

Context, tokens and memory — how a model reads you

Time: one class, Monday 9:00–2:00 · **Marks:** 20

Work from a terminal, and keep the session log. Everything you make today is committed and pushed before you leave — from now on the last part of every lab is `git add -A && git commit && git push`.

Work in `~/code/ITAS170/week03/`.

Get the starter files first: download `itas170-week03-student.zip` from the week 3 section on Moodle and unzip it into `~/code/ITAS170/week03/`. `ls` should show `tokens.py`, `cost.py`, `pricing.csv`, `sample.txt`, `README.txt` and a `templates/` folder.

How the day runs

Time		
9:00–9:40	Intro A	Tokens, the context window, and the price of a conversation
9:40–10:30	Session A — Parts 1-2	~50 min hands-on
10:30–11:00	Break	
11:00–11:40	Intro B	Prompt anatomy, templates with variables, the memory file
11:40–12:30	Session B — Parts 3-5	Instructor breaks for lunch around noon
12:30–2:00	Finish + demos	Help, then show+tell marking as you finish

You are marked in person, and demos run all day. 6 of the 20 marks are for explaining your work — this week's opening question is "why is the log so much more expensive than the essay?"

You have been talking to a model for two weeks without knowing what it sees. It does not see letters, or words, or files. It sees **tokens**, it has a **window** of them, and someone is **paying per million**. Today you count them, cost them, and learn to spend fewer of them for a better answer.

Part 1 — Count · 25 min

`tokens.py` needs a package that is not installed. `uv` fetches it into a cache the first time and reuses it after — no environment to make, no `pip`, nothing left behind in your project:

```
cd ~/code/ITAS170/week03
uv run --with tiktoken tokens.py sample.txt
```

file	chars	tokens	chars/token
sample.txt	140	46	3.04

Now see the pieces:

```
uv run --with tiktoken tokens.py --pieces sample.txt
```

The vertical bars show where one token ends and the next begins. Read it slowly: the same word splits differently depending on what is around it — `Str|aw|berry` at the start of a line is three pieces, `Strawberry` after a space is one; a timestamp is fifteen. **The model never sees letters.**

Then count real things — your two weeks of work, the log you hunted through, a Python file from ITAS 185, and any page of prose you like (a `.md` from this course is fine):

```
uv run --with tiktoken tokens.py ../week01/LOG.md ../week01/hunt.md \
  ../week01/access.log ~/code/ITAS185/week01/about_me.py \
  ../week02/gitlog.md
```

Write `tokens.md`: the table, then **three sentences** — which kind of text is the most expensive per character and why (look at the pieces of one log line), what the ratio for ordinary English prose is, and how the three strawberries in `sample.txt` split — and why differently.

Hand in: `tokens.md`.

Part 2 — Cost · 25 min

`pricing.csv` is a dated table of prices per million tokens for ten models. Nine you pay for by the token: the course's **yardstick**, Claude Opus 4.8 — every comparison this term is made against it — four more sizes of Claude, and four **open-weights** models from three Chinese labs (Kimi K3, DeepSeek V4 Pro and V4.1 Flash, GLM-5.3) whose weights are published on Hugging Face but which you rent here as hosted services. The tenth runs on your own

machine. Open the file and read the header: **the prices are data, not code**. They will be wrong within months; the arithmetic never will be.

```
uv run --with tiktoken cost.py 50000 20
```

That is "fifty thousand tokens of input, as a twenty-turn conversation". Read the two money columns against each other: a chat re-sends everything so far on **every turn**, which is why the right-hand column is not twenty times the left one but more — and why long sessions get slow and expensive at the same time. (One thing the table leaves out: vendors charge much less for context they have already seen — a cache read is a tenth of the input price on the yardstick — so a real bill for the same chat is lower. The shape holds; the number is the ceiling.)

Now cost your own repository:

```
uv run --with tiktoken cost.py --files ../week01/*.md ../week01/access.log \
  ../week02/*.md --turns 20
```

Write `cost.md`: both tables, then answer four questions in a sentence each. Which row would you use to (a) explain one command, (b) read your whole repository, (c) work with something private? (d) Which hosted row is cheapest per million output tokens — and name one thing the price column does **not** tell you about sending your text there (week 4 answers it). And **why does the local row say NO under fits?**, and what would you do about it?

Hand in: `cost.md`.

Part 3 — A template with variables · 30 min

A prompt you will type more than once should be a **file with holes in it**. Open `templates/explain-command.md` — two holes, `${COMMAND}` and `${LEVEL}`. The shell tool `envsubst` fills holes from environment variables (week 1's `export`, doing a new job):

```
mkdir -p prompts
COMMAND="sort -k10,10n access.log | tail -1" LEVEL="two weeks" \
  envsubst < templates/explain-command.md > prompts/explain-1.md
COMMAND="sort -k10,10n access.log | tail -1" LEVEL="ten years" \
  envsubst < templates/explain-command.md > prompts/explain-2.md
cat prompts/explain-1.md
```

Same command, two levels. Paste each filled prompt into your assistant in a **fresh** conversation each time, and read the two answers side by side.

Write `template.md`: the two `${LEVEL}` values you used, **three sentences** on what changed between the answers (length? vocabulary? did one break a rule in the template?), and **one rule you would add** to the template after seeing the answers. Then add it, and keep the improved template in your repo.

Hand in: `template.md` and the two files in `prompts/`.

Part 4 — The memory file, tested · 35 min · hands-only writing

Every fresh conversation starts with a blank desk. A **memory file** is the one page that goes on the desk first: the facts about your project that a tool cannot guess — where things are, what the conventions are, which commands work here, and what it must never touch. The open name for it is `AGENTS.md`; some tools read `CLAUDE.md` instead, so both exist and one points at the other.

Write `~/code/ITAS170/AGENTS.md` **yourself** — no assistant for this part; it is your description of your repository — with these four sections:

```
# ITAS170 – my course repository

## What this is
(two lines: the course, one folder per week, LOG.md in each)

## Conventions
(how LOG.md entries are formatted; where prompts and templates live;
 that nothing goes in git without a commit message that says what changed)

## Commands that work here
(the exact commands: uv run --with tiktoken week03/tokens.py FILE, etc.)

## Never
(never write to .env or secrets/; never paste keys; never run git push
 without being asked; anything else you would not want a tool to do)
```

Then make the pointer:

```
cd ~/code/ITAS170
printf 'Read AGENTS.md first. It is the memory file for this repository.\n' > CLAUDE.md
```

Now **test it**. Start a completely fresh assistant session with your repository open, and give it one small task: "Add a week-3 entry to `week03/LOG.md` in this repository's log format, then tell

me which file you read first to learn the format." Read what it did — did it use your format? did it name the memory file? did it touch anything under Never?

Write `memory-test.md`: the task you gave, what the tool did (in your words), whether it followed `AGENTS.md`, and one thing you changed in the file as a result. If your tool is a plain chat window with no file access, paste `AGENTS.md` in as the first message and run the same test — the lesson is identical.

The same idea, in the cloud. The class Project you joined in week 1 has instructions — a page the assistant reads before every chat inside that Project. That is a memory file too, living on the vendor's servers instead of in your repository. Open a chat inside the Project and give it the same task. Add two lines to `memory-test.md`: what did it already know about this course without reading a single file of yours, and what could it not do that the repository session could? (One of them has hands; one has your notes. Week 6 is about giving the one with hands better notes.)

Hand in: `AGENTS.md` and `CLAUDE.md` at the repository root, `week03/memory-test.md`.

Part 5 — Make it yours · 30 min

Three routes. Each is worth the full four marks.

Route A — your own template. A recurring task from your own life or another course, with **two or more variables**, in `templates/`. Fill it twice with `envsubst` and run both. (`templates/log-entry.md` shows the shape — yours must be for a different task.) Two sentences on what the variables buy you over retyping.

Route B — lazy versus structured. The same task twice: once as one careless line, once as a structured prompt (role, context, task, rules, format). Save both prompts, count their tokens with `tokens.py`, and judge the two answers on one criterion you name in advance. Which was cheaper per useful answer?

Route C — a memory file for something else you own. A folder of your own — a game project, a website from ITAS 191, a folder of notes — gets its own `AGENTS.md`, written by you, tested the same way as Part 4. Two sentences on what you left out on purpose.

Whichever route: commit and push everything before you demo.

```
cd ~/code/ITAS170
git add -A
git commit -m "Week 3: tokens, cost, templates, memory file"
git push
```

Hand in: the route's files, in the repo, pushed.

Marking — 20 marks

Component	Marks
C1 Count — the table, the pieces, three correct sentences	3
C2 Cost — both tables, four answers, the NO explained	2
C3 Template — two fills, the comparison, a rule added	2
C4 Memory file — four sections in your words, the pointer, an honest test, the Project compared	3
C5 Make it yours — Route A, B or C, working and explained	4
DU Demonstrates understanding — marked in person	6
	20

The **make it yours** criterion is worth **4 marks** — more than any single required part. DU is awarded from your show+tell demo, not from your files. Opening question: "why is the log so much more expensive than the essay?" Follow-up: "if I doubled the number of turns, would the cost double?"

Submitting

Your repository is the submission: everything above is committed and pushed by the end of Part 5. For the record, also zip `~/code/ITAS170/week03/` and upload it to the week 3 section on Moodle by **Sunday 4 October**.

On using AI for this lab

The assistant is the subject of Parts 3 and 4 and welcome everywhere else. One hands-only rule: **you write AGENTS.md yourself**. It is your account of your own repository, and a tool describing your repository to itself teaches you nothing. Every conversation goes in `LOG.md`. And the safety rule, again, because today you learn what an environment variable is for: **a key lives in a variable or a .env file that git ignores — never in a prompt, never in a template, never in a memory file.**

If something goes wrong

What you see	What it means	What to do
uv: command not found	uv is not installed for your account.	<code>curl -LsSf https://astral.sh/uv/install.sh</code> <code>less</code> — read it (week 1) — then run the installer, or tell the instructor: it should be on the image.
The first <code>uv run</code> sits for a while, then prints Downloaded tiktoken	It is fetching the package into uv's cache. Once.	Wait. Every later run is instant. No network? Use the instructor's table in Part 1 and say so in <code>tokens.md</code> .
<code>tokens.py: error: ...</code> No such file or directory	A path in the command is wrong — <code>../week01/LOG.md</code> is relative to where you are .	<code>pwd</code> . You should be in <code>~/code/ITAS170/week03</code> .
<code>envsubst: command not found</code>	The gettext-base package is missing.	<code>sudo apt install gettext-base</code> , or ask the instructor.
A hole in the filled prompt came out empty (for <code>.</code> where the level should be)	The variable was not set on the same line as <code>envsubst</code> , or its name is misspelt: an unset variable is replaced by nothing, silently.	Put <code>NAME="value"</code> immediately before <code>envsubst</code> on one line, as in the handout, and check the spelling against the template.
Your filled prompt has a <code>\$</code> in the command and <code>envsubst</code> ate it	<code>envsubst</code> replaces every <code>\$WORD</code> it sees, and a backslash in front does not protect it.	Name the variables you mean: <code>envsubst '\${COMMAND} \${LEVEL}'</code> — every other <code>\$</code> is left alone.
The assistant ignored <code>AGENTS.md</code> entirely	Not every tool reads it automatically; some read <code>CLAUDE.md</code> , some read nothing until told.	That is a result — record it. Paste the file in as the first message and re-run the test.
The assistant wrote to a file under <code>Never</code>	The memory file is advice to the tool, not a lock.	Record it in <code>memory-test.md</code> — this is the week-6 lesson arriving early. <code>git restore</code> the file.
<code>git push</code> is rejected	You committed before pulling on a second machine, or the remote moved.	<code>git pull</code> , then push. If you have never used a second machine, call the instructor.

If a number in your table looks nothing like the sample in this handout, that is interesting rather than wrong — different files have different shapes. Write down what you think explains it; that sentence is worth more than a matching number.